

Subscription-Based Authentication Guide

Complete guide for implementing subscription-based authentication with Lemon Squeezy and FastAPI.

Table of Contents

- 1. [Overview](#)
- 2. [Quick Start](#)
- 3. [Authentication Flow](#)
- 4. [Implementation Steps](#)
- 5. [Usage Examples](#)
- 6. [Database Schema](#)
- 7. [Frontend Integration](#)
- 8. [Testing](#)

Overview

This authentication system allows you to:

-  Protect API endpoints based on subscription status
-  Require specific subscription tiers for premium features
-  Use JWT tokens for session management
-  Support API key and license key authentication
-  Automatically sync with Lemon Squeezy webhooks
-  Implement rate limiting per subscription tier

Quick Start

1. Install Dependencies



bash

```
pip install fastapi httpx pyjwt python-jose[cryptography] python-multipart
```

2. Set Environment Variables

Add to your .env file:



env

Lemon Squeezy

LEMON_SQUEEZY_API_KEY=your_api_key

LEMON_SQUEEZY_WEBHOOK_SECRET=your_webhook_secret

JWT Authentication

JWT_SECRET_KEY=your-secret-key-change-in-production-use-long-random-string

Database

DATABASE_URL=postgresql://user:password@localhost/dbname

Generate a secure JWT secret:



bash

```
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

3. Update Your Main App



python

```
from fastapi import FastAPI
```

```
from lemon_squeezy_router import router as lemon_squeezy_router
```

```
from protected_endpoints import router as protected_router
```

```
app = FastAPI()
```

```
# Include routers
```

```
app.include_router(lemon_squeezy_router)
```

```
app.include_router(protected_router)
```

4. That's It!

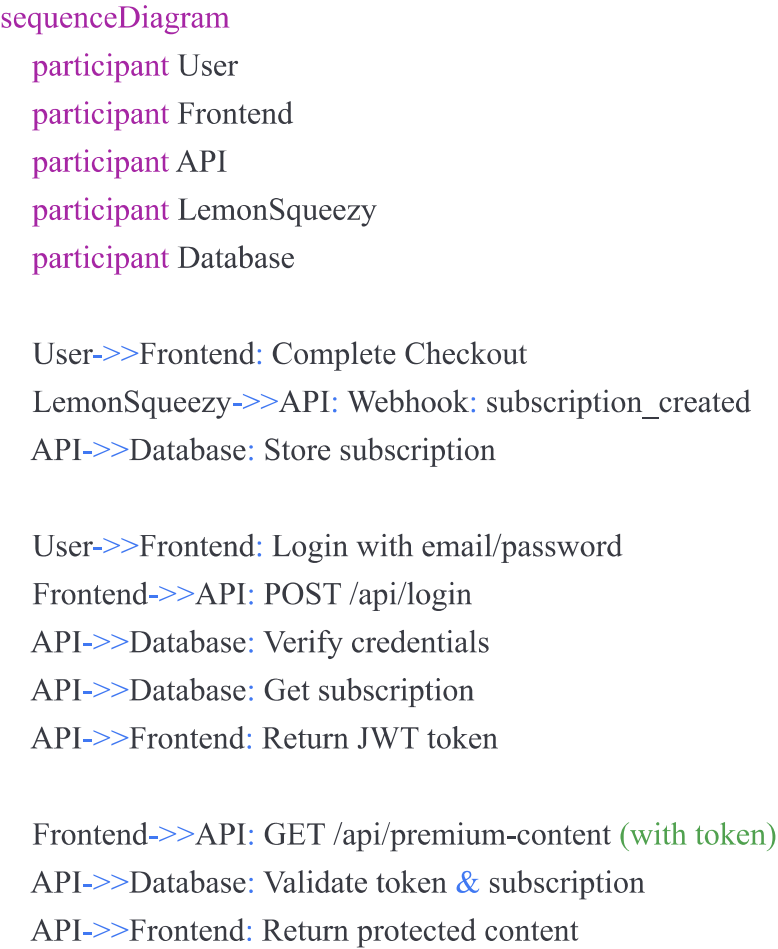
Your endpoints are now protected and will check subscription status automatically.

Authentication Flow

Standard Flow (After Checkout)



mermaid



License Key Flow (For Software Products)



mermaid

sequenceDiagram

participant App

participant API

participant LemonSqueezy

App->>API: Request with X-License-Key header

API->>LemonSqueezy: Validate license key

LemonSqueezy->>API: License valid/invalid

API->>App: Return data or 401

Implementation Steps

Step 1: Database Setup

Create tables to store users and subscriptions:



sql

-- Users table

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  user_id VARCHAR(255) UNIQUE NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  hashed_password VARCHAR(255) NOT NULL,  
  subscription_id VARCHAR(255),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Subscriptions table

```
CREATE TABLE subscriptions (  
  id SERIAL PRIMARY KEY,  
  lemon_squeezy_id VARCHAR(255) UNIQUE NOT NULL,  
  user_id INTEGER REFERENCES users(id),  
  customer_id VARCHAR(255) NOT NULL,  
  product_id VARCHAR(255) NOT NULL,  
  variant_id VARCHAR(255) NOT NULL,  
  status VARCHAR(50) NOT NULL,  
  card_brand VARCHAR(50),  
  card_last_four VARCHAR(4),  
  trial_ends_at TIMESTAMP,  
  renews_at TIMESTAMP,  
  ends_at TIMESTAMP,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Indexes for performance

```
CREATE INDEX idx_users_email ON users(email);  
CREATE INDEX idx_users_subscription ON users(subscription_id);  
CREATE INDEX idx_subscriptions_status ON subscriptions(status);  
CREATE INDEX idx_subscriptions_user ON subscriptions(user_id);
```

Step 2: Connect Webhooks to Database

Update `lemon_squeezy_router.py` webhook handlers to update your database:



python

```
async def handle_subscription_created(data: Dict[str, Any]):
    """Handle subscription created event"""
    sub_data = data.get("data", {}).get("attributes", {})
    subscription_id = data.get("data", {}).get("id")

    # Store in database
    async with get_db() as db:
        await db.execute("""
            INSERT INTO subscriptions (
                lemon_squeezy_id, customer_id, product_id,
                variant_id, status, renews_at
            ) VALUES ($1, $2, $3, $4, $5, $6)
            ON CONFLICT (lemon_squeezy_id)
            DO UPDATE SET status = $5, updated_at = NOW()
        """,
            subscription_id,
            sub_data.get("customer_id"),
            sub_data.get("product_id"),
            sub_data.get("variant_id"),
            sub_data.get("status"),
            sub_data.get("renews_at")
        )

    # Update user's subscription_id
    await db.execute("""
        UPDATE users
        SET subscription_id = $1
        WHERE email = (
            SELECT email FROM customers
            WHERE lemon_squeezy_id = $2
        )
    """, subscription_id, sub_data.get("customer_id"))

    logger.info(f"Subscription {subscription_id} created and stored")
```

Step 3: Implement Database Functions in auth.py

Replace the mock functions in auth.py:



python

```
from your_database import get_db # Your database connection
```

```
async def get_user_from_db(user_id: str) -> Optional[dict]:
    """Get user from database"""
    async with get_db() as db:
        result = await db.fetchone(
            "SELECT * FROM users WHERE user_id = $1",
            user_id
        )
        return dict(result) if result else None

async def get_subscription_from_db(subscription_id: str) -> Optional[dict]:
    """Get subscription from database"""
    async with get_db() as db:
        result = await db.fetchone(
            "SELECT * FROM subscriptions WHERE lemon_squeezy_id = $1",
            subscription_id
        )
        return dict(result) if result else None
```

Step 4: Protect Your Endpoints

Use the dependencies in your routes:



python

```

from fastapi import APIRouter, Depends
from auth import get_current_active_subscriber, User

router = APIRouter()

@router.get("/protected")
async def protected_endpoint(
    user: User = Depends(get_current_active_subscriber)
):
    """Only accessible with active subscription"""
    return {"message": "Success", "user": user.user_id}

```

Usage Examples

Protecting Endpoints

Basic Authentication (Any logged-in user)



python

```

from auth import get_current_user

@app.get("/profile")
async def profile(user: User = Depends(get_current_user)):
    """Requires login, but not subscription"""
    return {"user": user.user_id}

```

Subscription Required



python


```
from auth import get_current_active_subscriber
```

```
@app.get("/premium")
```

```
async def premium(user: User = Depends(get_current_active_subscriber)):  
    """Requires active subscription"""  
    return {"content": "Premium content"}
```

Specific Plan Required



python

```
from auth import get_current_active_subscriber, require_plan
```

```
@app.get("/pro-feature")
```

```
async def pro_feature(  
    user: User = Depends(get_current_active_subscriber),  
    _: None = Depends(require_plan(["variant_12345", "variant_67890"])),  
):  
    """Requires Pro or Enterprise plan"""  
    return {"feature": "Pro feature"}
```

Rate Limited by Plan



python

```
from auth import rate_limiter
```

```
@app.get("/api-call")
```

```
async def api_call(user: User = Depends(rate_limiter.check_limit)):  
    """Different rate limits based on plan"""  
    return {"data": "..."} 
```

Getting Variant IDs from Lemon Squeezy

1. Go to your Lemon Squeezy dashboard
2. Navigate to Products → Select your product
3. Click on a variant
4. Copy the variant ID from the URL or the variant details

Example: <https://app.lemonsqueezy.com/products/123/variants/456> The variant ID is 456

Database Schema

Recommended Database Schema



python

```
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey
from sqlalchemy.ext.declarative import declarative_base
from datetime import datetime
```

```
Base = declarative_base()
```

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    user_id = Column(String, unique=True, nullable=False)
    email = Column(String, unique=True, nullable=False)
    hashed_password = Column(String, nullable=False)
    subscription_id = Column(String, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)
```

```
class Subscription(Base):
    __tablename__ = "subscriptions"

    id = Column(Integer, primary_key=True)
    lemon_squeezy_id = Column(String, unique=True, nullable=False)
    user_id = Column(Integer, ForeignKey("users.id"))
    status = Column(String, nullable=False)
    product_id = Column(String, nullable=False)
    variant_id = Column(String, nullable=False)
    renews_at = Column(DateTime, nullable=True)
    ends_at = Column(DateTime, nullable=True)
```

Frontend Integration

Login and Store Token



javascript

```
// Login
async function login(email, password) {
  const response = await fetch('https://api.example.com/api/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password })
  });

  if (!response.ok) {
    throw new Error('Login failed');
  }

  const { access_token, expires_in } = await response.json();

  // Store token
  localStorage.setItem('token', access_token);
  localStorage.setItem('token_expiry', Date.now() + (expires_in * 1000));

  return access_token;
}
```

Make Authenticated Requests



javascript

// API client with auto-retry

```
class APIClient {
  constructor(baseUrl) {
    this.baseUrl = baseUrl;
  }

  getToken() {
    return localStorage.getItem('token');
  }

  isTokenExpired() {
    const expiry = localStorage.getItem('token_expiry');
    return !expiry || Date.now() > parseInt(expiry);
  }

  async request(endpoint, options = {}) {
    if (this.isTokenExpired()) {
      // Redirect to login or refresh token
      window.location.href = '/login';
      return;
    }

    const token = this.getToken();
    const response = await fetch(`${this.baseUrl}${endpoint}`, {
      ...options,
      headers: {
        ...options.headers,
        'Authorization': `Bearer ${token}`
      }
    });

    if (response.status === 401) {
      // Token invalid - redirect to login
      window.location.href = '/login';
      return;
    }

    if (response.status === 403) {
      const error = await response.json();
      if (error.detail.includes('subscription')) {
        // No subscription - redirect to pricing

```

```
    window.location.href = '/pricing';
    return;
  }
}

return response;
}

async getPremiumContent() {
  const response = await this.request('/api/premium-content');
  return response?.json();
}
}

// Usage
const api = new APIClient('https://api.example.com');
const content = await api.getPremiumContent();
```

React Hook Example



javascript

```
import { useState, useEffect } from 'react';
```

```
function useAuth() {  
  const [user, setUser] = useState(null);  
  const [loading, setLoading] = useState(true);  
  
  useEffect(() => {  
    async function checkAuth() {  
      const token = localStorage.getItem('token');  
      if (!token) {  
        setLoading(false);  
        return;  
      }  
  
      try {  
        const response = await fetch('https://api.example.com/api/profile', {  
          headers: { 'Authorization': `Bearer ${token}` }  
        });  
  
        if (response.ok) {  
          const userData = await response.json();  
          setUser(userData);  
        }  
      } catch (error) {  
        console.error('Auth check failed:', error);  
      } finally {  
        setLoading(false);  
      }  
    }  
  
    checkAuth();  
  }, []);  
  
  return { user, loading };  
}
```

```
// Protected component
```

```
function ProtectedPage() {  
  const { user, loading } = useAuth();  
  
  if (loading) return <div>Loading...</div>;
```

```
if (!user) return <Navigate to="/login" />;

return <div>Welcome, {user.email}!</div>;
}
```

Testing

Test with curl



bash

```
# Login
TOKEN=$(curl -X POST http://localhost:8000/api/login \
-H "Content-Type: application/json" \
-d '{"email":"user@example.com","password":"password"}' \
| jq -r '.access_token')

# Call protected endpoint
curl http://localhost:8000/api/premium-content \
-H "Authorization: Bearer $TOKEN"

# Expected success: 200 OK with content
# No token: 401 Unauthorized
# No subscription: 403 Forbidden
```

Test Subscription States



python

```

import pytest
from fastapi.testclient import TestClient

def test_no_auth():
    """Test accessing protected endpoint without token"""
    response = client.get("/api/premium-content")
    assert response.status_code == 401

def test_no_subscription(auth_token_no_sub):
    """Test with valid token but no subscription"""
    response = client.get(
        "/api/premium-content",
        headers={"Authorization": f"Bearer {auth_token_no_sub}"})
    assert response.status_code == 403
    assert "subscription" in response.json()["detail"].lower()

def test_with_subscription(auth_token_with_sub):
    """Test with valid token and active subscription"""
    response = client.get(
        "/api/premium-content",
        headers={"Authorization": f"Bearer {auth_token_with_sub}"})
    assert response.status_code == 200

```

Common Patterns

Pattern 1: User Registers → Buys → Gets Access



python

1. User registers (no subscription yet)

```
@app.post("/register")
async def register(email: str, password: str):
    user = await create_user(email, password)
    return {"user_id": user.id, "message": "Please purchase a subscription"}
```

2. User buys subscription (Lemon Squeezy checkout)

3. Webhook updates database with subscription

4. User logs in and gets token with subscription access

5. User can now access protected endpoints

Pattern 2: Free Trial → Premium



python

```
@app.get("/content")
async def get_content(user: User = Depends(get_current_active_subscriber)):
    # User can access during trial (status: "on_trial")
    # or active subscription (status: "active")
    return {"content": "Premium content"}
```

Pattern 3: Tiered Features



python

Basic tier

```
@app.get("/basic-feature")
async def basic(user: User = Depends(get_current_active_subscriber)):
    return {"feature": "basic"}
```

Pro tier

```
@app.get("/pro-feature")
async def pro(
    user: User = Depends(get_current_active_subscriber),
    _: None = Depends(require_plan(["pro_variant", "enterprise_variant"])),
):
    return {"feature": "pro"}
```

Enterprise tier

```
@app.get("/enterprise-feature")
async def enterprise(
    user: User = Depends(get_current_active_subscriber),
    _: None = Depends(require_plan(["enterprise_variant"])),
):
    return {"feature": "enterprise"}
```

Troubleshooting

401 Unauthorized

- Check token is being sent in Authorization header
- Verify token hasn't expired (default: 30 minutes)
- Ensure JWT_SECRET_KEY matches between token creation and validation

403 Forbidden "No subscription found"

- Verify webhook successfully updated database
- Check subscription_id is set in users table
- Confirm subscription status is "active" or "on_trial"

403 Forbidden "Subscription is cancelled"

- User cancelled subscription
- Redirect to billing page or show upgrade modal
- Subscription remains active until period end

Subscription not syncing

- Verify webhook endpoint is receiving events (check logs)
- Confirm webhook signature verification is passing

- Check database update queries in webhook handlers

Best Practices

1. **Always verify webhook signatures** - Prevents unauthorized database updates
2. **Use HTTPS in production** - Required for webhooks and secure token transmission
3. **Implement token refresh** - Add refresh token endpoint for better UX
4. **Cache subscription checks** - Reduce database queries with Redis cache
5. **Handle expired subscriptions gracefully** - Show friendly upgrade messages
6. **Log authentication attempts** - Monitor for suspicious activity
7. **Test subscription lifecycle** - Test trial → active → cancelled → expired
8. **Document your variant IDs** - Keep a reference of which IDs match which plans

Next Steps

- ☐ Implement password hashing (use bcrypt or argon2)
- ☐ Add email verification
- ☐ Implement password reset
- ☐ Add refresh token endpoint
- ☐ Set up Redis for rate limiting
- ☐ Add monitoring and alerts
- ☐ Implement subscription change notifications
- ☐ Create customer dashboard
- ☐ Add usage analytics per user
- ☐ Set up automated tests

Resources

- [FastAPI Security Documentation](#)
- [Lemon Squeezy API Docs](#)
- [JWT.io](#) - JWT debugger
- [OWASP API Security](#)